

2.14 string.h

The string header provides many functions useful for manipulating strings (character arrays).

Macros:

```
NULL
```

Variables:

```
typedef size_t
```

Functions:

```
memchr();  
memcmp();  
memcpy();  
memmove();  
memset();  
strcat();  
strncat();  
strchr();  
strcmp();  
strncmp();  
strcoll();  
strcpy();  
strncpy();  
strcspn();  
strerror();  
strlen();  
strpbrk();  
strrchr();  
strspn();  
strstr();  
strtok();  
strxfrm();
```

2.14.1 Variables and Definitions

size_t is the unsigned integer result of the sizeof keyword.

NULL is the value of a null pointer constant.

2.14.2 memchr

Declaration:

```
void *memchr(const void *str, int c, size_t n);
```

Searches for the first occurrence of the character *c* (an **unsigned char**) in the first *n* bytes of the string pointed to by the argument *str*.

Returns a pointer pointing to the first matching character, or null if no match was found.

2.14.3 memcmp

Declaration:

```
int memcmp(const void *str1, const void *str2, size_t n);
```

Compares the first n bytes of *str1* and *str2*. Does not stop comparing even after the null character (it always checks n characters).

Returns zero if the first n bytes of *str1* and *str2* are equal. Returns less than zero or greater than zero if *str1* is less than or greater than *str2* respectively.

2.14.4 memcpy

Declaration:

```
void *memcpy(void *str1, const void *str2, size_t n);
```

Copies n characters from *str2* to *str1*. If *str1* and *str2* overlap the behavior is undefined.

Returns the argument *str1*.

2.14.5 memmove

Declaration:

```
void *memmove(void *str1, const void *str2, size_t n);
```

Copies n characters from *str2* to *str1*. If *str1* and *str2* overlap the information is first completely read from *str1* and then written to *str2* so that the characters are copied correctly.

Returns the argument *str1*.

2.14.6 memset

Declaration:

```
void *memset(void *str, int c, size_t n);
```

Copies the character c (an `unsigned char`) to the first n characters of the string pointed to by the argument *str*.

The argument *str* is returned.

2.14.7 strcat

Declaration:

```
char *strcat(char *str1, const char *str2);
```

Appends the string pointed to by *str2* to the end of the string pointed to by *str1*. The terminating null character of *str1* is overwritten. Copying stops once the terminating null character of *str2* is copied. If overlapping occurs, the result is undefined.

The argument *str1* is returned.

2.14.8 strncat

Declaration:

```
char *strncat(char *str1, const char *str2, size_t n);
```

Appends the string pointed to by *str2* to the end of the string pointed to by *str1* up to *n* characters long. The terminating null character of *str1* is overwritten. Copying stops once *n* characters are copied or the terminating null character of *str2* is copied. A terminating null character is always appended to *str1*. If overlapping occurs, the result is undefined.

The argument *str1* is returned.

2.14.9 strchr

Declaration:

```
char *strchr(const char *str, int c);
```

Searches for the first occurrence of the character *c* (an unsigned char) in the string pointed to by the argument *str*. The terminating null character is considered to be part of the string.

Returns a pointer pointing to the first matching character, or null if no match was found.

2.14.10 strcmp

Declaration:

```
int strcmp(const char *str1, const char *str2);
```

Compares the string pointed to by *str1* to the string pointed to by *str2*.

Returns zero if *str1* and *str2* are equal. Returns less than zero or greater than zero if *str1* is less than or greater than *str2* respectively.

2.14.11 strncmp

Declaration:

```
int strncmp(const char *str1, const char *str2, size_t n);
```

Compares at most the first n bytes of $str1$ and $str2$. Stops comparing after the null character.

Returns zero if the first n bytes (or null terminated length) of $str1$ and $str2$ are equal. Returns less than zero or greater than zero if $str1$ is less than or greater than $str2$ respectively.

2.14.12 strcoll

Declaration:

```
int strcoll(const char *str1, const char *str2);
```

Compares string $str1$ to $str2$. The result is dependent on the `LC_COLLATE` setting of the location.

Returns zero if $str1$ and $str2$ are equal. Returns less than zero or greater than zero if $str1$ is less than or greater than $str2$ respectively.

2.14.13 strcpy

Declaration:

```
char *strcpy(char *str1, const char *str2);
```

Copies the string pointed to by $str2$ to $str1$. Copies up to and including the null character of $str2$. If $str1$ and $str2$ overlap the behavior is undefined.

Returns the argument $str1$.

2.14.14 strncpy

Declaration:

```
char *strncpy(char *str1, const char *str2, size_t n);
```

Copies up to n characters from the string pointed to by $str2$ to $str1$. Copying stops when n characters are copied or the terminating null character in $str2$ is reached. If the null character is reached, the null characters are continually copied to $str1$ until n characters have been copied.

Returns the argument $str1$.

2.14.15 strcspn

Declaration:

```
size_t strcspn(const char *str1, const char *str2);
```

Finds the first sequence of characters in the string *str1* that does not contain any character specified in *str2*.

Returns the length of this first sequence of characters found that do not match with *str2*.

2.14.16 strerror

Declaration:

```
char *strerror(int errnum);
```

Searches an internal array for the error number *errnum* and returns a pointer to an error message string.

Returns a pointer to an error message string.

2.14.17 strlen

Declaration:

```
size_t strlen(const char *str);
```

Computes the length of the string *str* up to but not including the terminating null character.

Returns the number of characters in the string.

2.14.18 strpbrk

Declaration:

```
char *strpbrk(const char *str1, const char *str2);
```

Finds the first character in the string *str1* that matches any character specified in *str2*.

A pointer to the location of this character is returned. A null pointer is returned if no character in *str2* exists in *str1*.

Example:

```
#include<string.h>
#include<stdio.h>

int main(void)
{
    char string[]="Hi there, Chip!";
    char *string_ptr;

    while((string_ptr=strpbrk(string, " "))!=NULL)
```

```

    *string_ptr='-';

    printf("New string is \"%s\".\n",string);
    return 0;
}

```

The output should result in every space in the string being converted to a dash (-).

2.14.19 strrchr

Declaration:

```
char *strrchr(const char *str, int c);
```

Searches for the last occurrence of the character *c* (an `unsigned char`) in the string pointed to by the argument *str*. The terminating null character is considered to be part of the string.

Returns a pointer pointing to the last matching character, or null if no match was found.

2.14.20 strspn

Declaration:

```
size_t strspn(const char *str1, const char *str2);
```

Finds the first sequence of characters in the string *str1* that contains any character specified in *str2*.

Returns the length of this first sequence of characters found that match with *str2*.

Example:

```

#include<string.h>
#include<stdio.h>

int main(void)
{
    char string[]="7803 Elm St.";

    printf("The number length is %d.\n",strspn(string,"1234567890"));

    return 0;
}

```

The output should be: The number length is 4.

2.14.21 strstr

Declaration:

```
char *strstr(const char *str1, const char *str2);
```

Finds the first occurrence of the entire string *str2* (not including the terminating null character) which appears in the string *str1*.

Returns a pointer to the first occurrence of *str2* in *str1*. If no match was found, then a null pointer is returned. If *str2* points to a string of zero length, then the argument *str1* is returned.

2.14.22 strtok

Declaration:

```
char *strtok(char *str1, const char *str2);
```

Breaks string *str1* into a series of tokens. If *str1* and *str2* are not null, then the following search sequence begins. The first character in *str1* that does not occur in *str2* is found. If *str1* consists entirely of characters specified in *str2*, then no tokens exist and a null pointer is returned. If this character is found, then this marks the beginning of the first token. It then begins searching for the next character after that which is contained in *str2*. If this character is not found, then the current token extends to the end of *str1*. If the character is found, then it is overwritten by a null character, which terminates the current token. The function then saves the following position internally and returns.

Subsequent calls with a null pointer for *str1* will cause the previous position saved to be restored and begins searching from that point. Subsequent calls may use a different value for *str2* each time.

Returns a pointer to the first token in *str1*. If no token is found then a null pointer is returned.

Example:

```
#include<string.h>
#include<stdio.h>

int main(void)
{
    char search_string[]="Woody Norm Cliff";
    char *array[50];
    int loop;

    array[0]=strtok(search_string, " ");
    if(array[0]==NULL)
    {
        printf("No test to search.\n");
        exit(0);
    }

    for(loop=1;loop<50;loop++)
    {
        array[loop]=strtok(NULL, " ");
        if(array[loop]==NULL)
            break;
    }

    for(loop=0;loop<50;loop++)
    {
        if(array[loop]==NULL)
            break;
    }
}
```

```
        printf("Item #%d is %s.\n",loop,array[loop]);  
    }  
  
    return 0;  
}
```

This program replaces each space into a null character and stores a pointer to each substring into the array. It then prints out each item.

2.14.23 strxfrm

Declaration:

```
size_t strxfrm(char *str1, const char *str2, size_t n);
```

Transforms the string *str2* and places the result into *str1*. It copies at most *n* characters into *str1* including the null terminating character. The transformation occurs such that `strcmp` applied to two separate converted strings returns the same value as `strcmp` applied to the same two strings. If overlapping occurs, the result is undefined.

Returns the length of the transformed string (not including the null character).